Azure Service Bus – Sending Messages to a Topic and Subscriptions

1. Overview

In Azure Service Bus, a **Topic** is used for **publish-subscribe messaging**.

A sender publishes a message to a topic, and Azure Service Bus delivers a copy of that message to the subscriptions associated with that topic.

Basic Flow

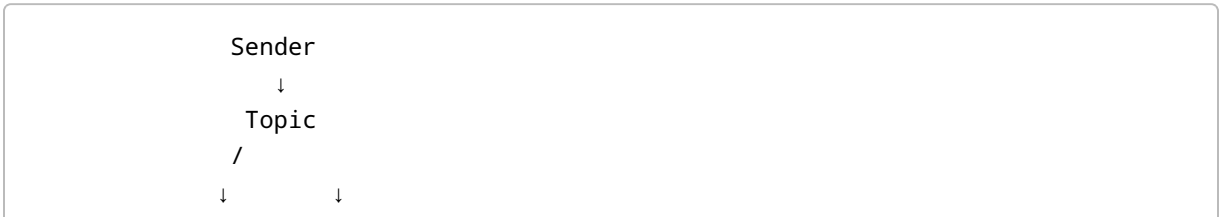
If a topic has multiple subscriptions, a published message can be delivered to each subscription.

---

2. Topic vs Queue

A **Queue** is generally used for point-to-point messaging.

A **Topic** is used for publish-subscribe messaging.



```
        S1       S2
         ↓       ↓
      Receiver Receiver
```

Important Difference

Queue:

One message is normally processed by one competing receiver.

Topic:

One published message can be delivered to multiple subscriptions.

---

3. Example

Suppose we have:

```
Topic: FirstTopic

Subscriptions:
    S1
    S2
```

Initially:

```
S1 → 0 messages
S2 → 0 messages
```

Now the application publishes:

```
Hello from Visual Studio
```

to the topic.

Azure Service Bus delivers the message to both subscriptions:

```
Topic: FirstTopic
        |
        ├── S1 → "Hello from Visual Studio"
```

```
        │
        └── S2 → "Hello from Visual Studio"
```

Therefore:

```
S1 → 1 message
S2 → 1 message
```

---

4. Important Concept

When sending a message to a topic, we **do not send the message directly to a subscription**.

We send the message to the **topic**.

Azure Service Bus then handles delivery to the subscriptions.

```
Application
     │
     │ Send Message
     ↓
   Topic
     │
     ├──────────→ Subscription S1
     │
     └──────────→ Subscription S2
```

---

5. Required NuGet Package

To work with Azure Service Bus from a .NET application, install the required Azure Service Bus NuGet package:

```
Azure.Messaging.ServiceBus
```

This package provides classes such as:

```
ServiceBusClient
ServiceBusSender
```

```
ServiceBusReceiver
ServiceBusMessage
```

---

6. Connection String

To connect to the Service Bus namespace, we need a connection string.

A connection string can be obtained from the Azure Portal through the Service Bus entity's **Shared access policies**.

The connection string is used to create the `ServiceBusClient`.

Example:

```
string connectionString = "<connection-string>";
```

Important

The connection string should be treated as a **secret**.

Do not hard-code real connection strings in source code that is committed to Git or shared publicly.

For production applications, prefer secure configuration such as:

- Azure Key Vault
- Managed Identity
- Environment variables
- Application configuration/secrets management

---

7. Create ServiceBusClient

First, create a `ServiceBusClient`.

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);
```

The `ServiceBusClient` is used to communicate with Azure Service Bus.

---

8. Create a Sender for the Topic

Suppose our topic name is:

```
first-topic
```

Create a sender:

```
ServiceBusSender sender =
    client.CreateSender("first-topic");
```

The sender is responsible for sending messages to the topic.

---

9. Create a ServiceBusMessage

Create the message using `ServiceBusMessage`.

```
ServiceBusMessage message =
    new ServiceBusMessage("Hello from Visual Studio");
```

Here:

```
"Hello from Visual Studio"
```

is the message body.

---

10. Send the Message

Send the message using:

```
await sender.SendMessageAsync(message);
```

The complete basic flow is:

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);

ServiceBusSender sender =
    client.CreateSender("first-topic");

ServiceBusMessage message =
    new ServiceBusMessage("Hello from Visual Studio");

await sender.SendMessageAsync(message);
```

---

11. What Happens After Sending?

Suppose our topic has two subscriptions:

```
FirstTopic
    |
    ├── S1
    └── S2
```

We send:

```
Hello from Visual Studio
```

to the topic.

Azure Service Bus creates/delivers a copy for each applicable subscription:

```
                FirstTopic
                    |
            "Hello from Visual Studio"
                 /
                ↓         ↓
               S1         S2
                |         |
                ↓         ↓
             Message   Message
```

Therefore:

```
S1 → 1 message
S2 → 1 message
```

---

12. Why Does Each Subscription Get the Message?

A topic follows the **publish-subscribe pattern**.

The publisher does not need to know which applications are consuming the message.

The publisher simply publishes:

```
Message → Topic
```

The topic distributes the message to its subscriptions.

Each subscription acts like its own message store.

For example:

Both applications can independently process the message.

---

13. Real-Time Example

Imagine an e-commerce application.

When a customer places an order, the application publishes:

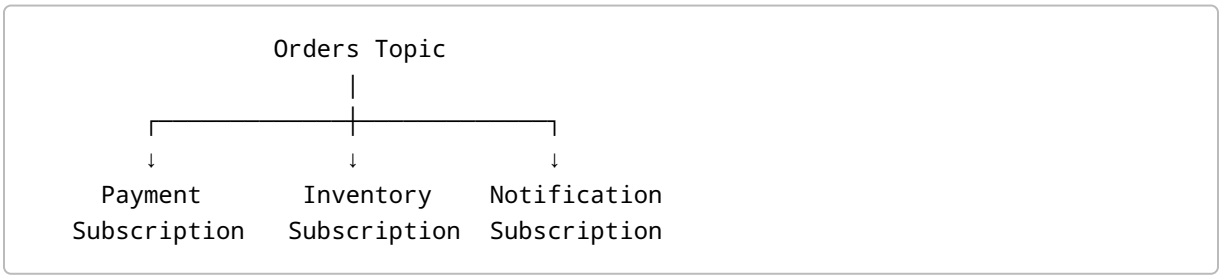
```
OrderId = 1001
Customer = Puneeth
Amount = ₹5000
```

to an `Orders` topic.

The topic has:

```
Orders Topic
     |
     ├── Payment Subscription
     |
     ├── Inventory Subscription
     |
     └── Notification Subscription
```

The same published event can reach all three subscriptions.



Each subscriber can perform a different operation.

Payment

Processes the payment.

Inventory

Updates product stock.

Notification

Sends an order confirmation.

This is one of the major use cases of Azure Service Bus Topics.

---

14. Queue vs Topic – Interview Comparison

Feature	Queue	Topic
Messaging pattern	Point-to-point	Publish-subscribe
Multiple consumers	Competing consumers	Multiple subscriptions
Sender sends to	Queue	Topic
Message copied to multiple subscriptions	No	Yes
Useful for	Work distribution	Event broadcasting
Example	Process one order	Notify multiple systems about an order

---

15. Important Terminology

Publisher / Sender

The application that sends the message.

```
Publisher → Topic
```

Topic

The destination where the message is published.

```
Publisher
    ↓
Topic
```

Subscription

A named, durable view of messages published to a topic.

```
Topic
 ├── S1
```

```
  ├── S2
  └── S3
```

Subscriber / Receiver

The application that receives messages from a subscription.

```
Subscription
      ↓
Subscriber
```

---

16. Topic Message Flow

Remember this flow:

```
Publisher
    |
    | SendMessageAsync()
    ↓
  Topic
    |
    ├──────────→ Subscription S1
    |                  ↓
    |               Receiver
    |
    └──────────→ Subscription S2
                       ↓
                    Receiver
```

The publisher only needs to know the **topic**.

It does not need to directly send the message to S1 or S2.

---

17. Example from the Video

Initially, the topic had two subscriptions:

```
Topic: FirstTopic
```

```
S1 → 0 messages
S2 → 0 messages
```

The application created a message:

```
Message = "Hello..."
```

Then created a sender for the topic:

```
ServiceBusSender sender =
    client.CreateSender(topicName);
```

The message was sent:

```
await sender.SendMessageAsync(message);
```

After sending, the Azure Portal was checked.

Subscription S1

```
S1 → 1 message
```

The message body matched the message sent from Visual Studio.

Subscription S2

```
S2 → 1 message
```

The same message was also available in S2.

Therefore:

This demonstrates the publish-subscribe behavior of Azure Service Bus Topics.

---

18. Important Interview Question

Q: If I send one message to an Azure Service Bus Topic with two subscriptions, how many messages will be available?

Answer:

If the message matches both subscriptions, each subscription receives its own copy of the message.

For example:

```
Topic
 ├── S1 → 1 message
 └── S2 → 1 message
```

So there is one message available in each subscription.

---

19. Important Interview Question

Q: Do we send a message directly to a subscription?

Answer:

No.

The application sends the message to the **topic**. Azure Service Bus then makes the message available to the appropriate subscriptions.

```
Sender → Topic → Subscriptions
```

---

20. Important Interview Question

Q: What is the main purpose of a Topic?

Answer:

A Service Bus Topic is used for **publish-subscribe messaging**, where a publisher sends a message once and multiple subscriptions can receive that message.

---

21. Important Interview Question

Q: What happens if a topic has three subscriptions?

Suppose:

```
Topic
 ├── S1
 ├── S2
 └── S3
```

The publisher sends:

```
Message A
```

If the message matches all three subscriptions:

```
S1 → Message A
S2 → Message A
S3 → Message A
```

Each subscription can independently process the message.

---

22. Key Points to Remember

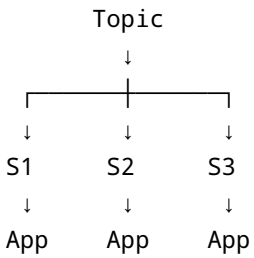
- **Topic = Publish-Subscribe pattern**
- A sender publishes a message to a topic.
- A topic can have multiple subscriptions.
- Each applicable subscription receives its own copy of the message.
- The sender does not directly send messages to subscriptions.
- `ServiceBusClient` creates the connection.
- `ServiceBusSender` sends messages.
- `ServiceBusMessage` represents the message.
- `SendMessageAsync()` sends the message.
- Each subscription can have its own receiver.
- Subscriptions can be used by different applications for different purposes.

Quick Memory Trick

```
QUEUE

Sender
  ↓
Queue
  ↓
Receiver


TOPIC
```



One-Line Interview Answer

> **Azure Service Bus Topic follows the publish-subscribe pattern: a publisher sends a message to the topic, and the message is made available to each matching subscription independently.**